StudyAI — Master Architecture

Virtual Teaching Assistant · CP5105 Final Year Project · three views of one system, from infrastructure down to a single chat turn

Developed & deployed on AWS
CI builds the container images into Amazon ECR and runs them on Amazon EC2 in us-east-1; the Postgres + pgvector store lives on the same host.

The three views are the same system at three magnifications. View 1 shows where it runs; one box in it — the backend service — is the whole of View 2. View 2 shows how the modules wire together; one module in it — server/chat.js — is the whole of View 3. Each view keeps its own numbering; the two grey arrows down the left margin mark where one view expands into the next.

Four use cases, lit up across all three views — each is a drawio **layer** (View menu → Layers, or Ctrl/Cmd+Shift+L). Show one at a time for the clearest path, or all four to see where they share infrastructure. Rings mark the components on the path, the translucent band follows its wires, and the small dots are that use case's own steps.

● **S1** · Student · **Ask the TA about course material**

1. Types a question in the chat
2. DNS → CDN → load balancer
3. Session checked; router hands it to server/chat.js
4. Retrieval: embed the question, search pgvector, optionally compress
5. Persona wraps the context
6. Agent loop calls the model
7. Tools run as needed — progress, practice question, docs
8. The answer streams back, token by token

● **S2** · Student · **Take an adaptive quiz**

1. Starts a ten-question sitting
2. Request reaches server/quiz.js
3. Fuzzy engine sets the target Bloom level and complexity
4. Question from the bank; on a miss a model generates one, then it is cached
5. Every answer is persisted
6. On completion, one coaching note for the whole sitting

● **I1** · Instructor · **Analyze this course**

1. Clicks Analyze on a course
2. Role gate confirms instructor; server/stats.js takes it
3. Class signals recomputed from the database
4. Live web search for subject trends, in parallel
5. One prompt to the model; the note streams back

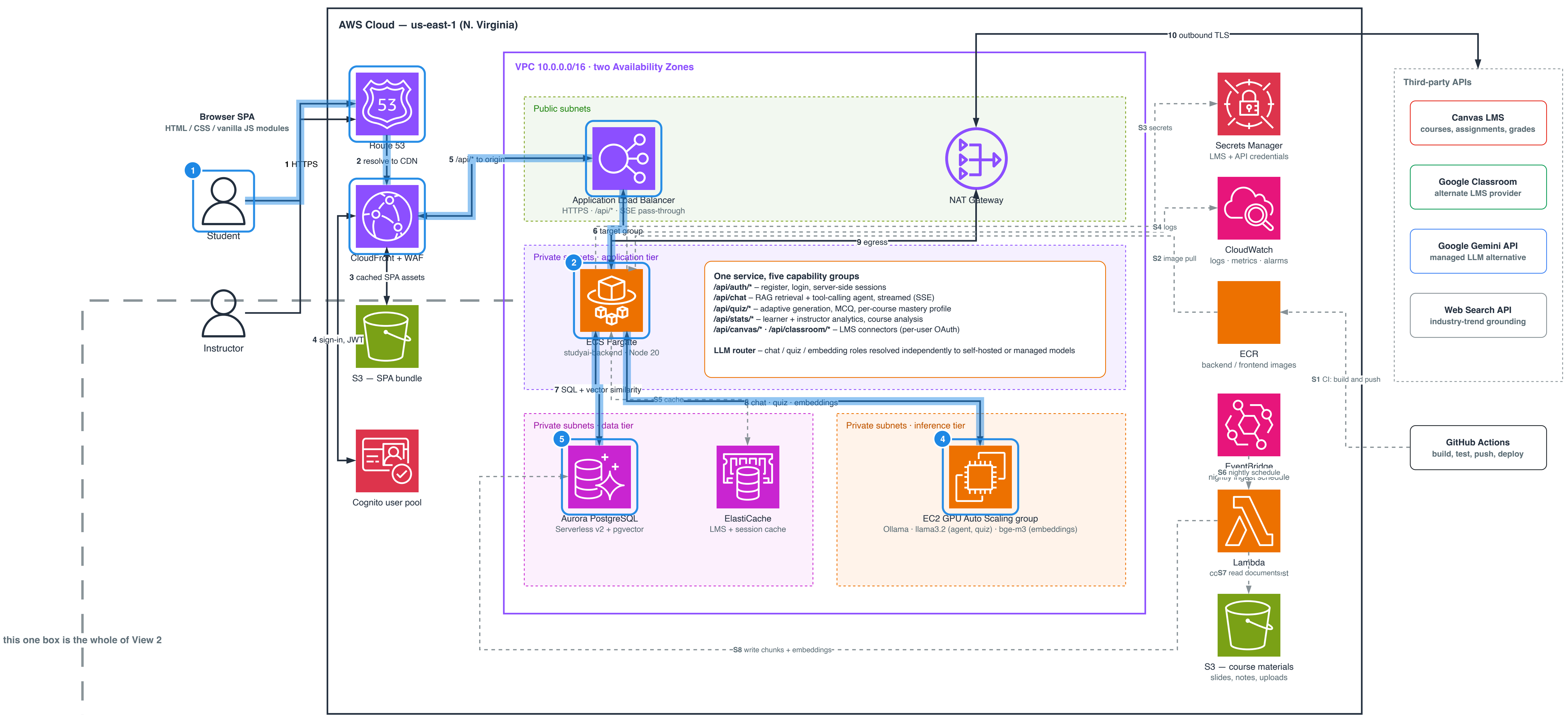
● **I2** · Instructor · **Connect Canvas and pull courses**

1. Signs in with Canvas
2. OAuth token stored per user
3. Courses, assignments and grades fetched from Canvas
4. Normalised and rendered; calendar and dashboards read the same data

View 3 · Not on this path — syncing the LMS involves no AI.

View 1 · Deployment

Where the system runs. AWS services, the request path 1–10, and the supporting traffic S1–S8.



AWS Cloud — us-east-1 (N. Virginia)

Browser SPA
HTML / CSS / vanilla JS modules

1 Student

Instructor

1 HTTPS

Route 53

2 resolve to CDN

CloudFront + WAF

3 cached SPA assets

4 sign-in, JWT

S3 — SPA bundle

Cognito user pool

VPC 10.0.0.0/16 · two Availability Zones

Public subnets

5 /api/* to origin

Application Load Balancer
HTTPS · /api/* · SSE pass-through

NAT Gateway

10 outbound TLS

6 target group

9 egress

Private subnets · application tier

2 ECS Fargate
studyai-backend · Node 20

One service, five capability groups
/api/auth/* – register, login, server-side sessions
/api/chat – RAG retrieval + tool-calling agent, streamed (SSE)
/api/quiz/* – adaptive generation, MCQ, per-course mastery profile
/api/stats/* – learner + instructor analytics, course analysis
/api/canvas/* · /api/classroom/* – LMS connectors (per-user OAuth)

LLM router – chat / quiz / embedding roles resolved independently to self-hosted or managed models

7 SQL + vector similarity

S5 cache

8 chat · quiz · embeddings

Private subnets · data tier

5 Aurora PostgreSQL
Serverless v2 + pgvector

ElastiCache
LMS + session cache

Private subnets · inference tier

4 EC2 GPU Auto Scaling group
Ollama · llama3.2 (agent, quiz) · bge-m3 (embeddings)

S3 secrets

Secrets Manager
LMS + API credentials

S4 logs

CloudWatch
logs · metrics · alarms

S2 image pull

ECR
backend / frontend images

S1 CI: build and push

EventBridge

S6 nightly schedule

Lambda

S7 read documents

S3 — course materials
slides, notes, uploads

S8 write chunks + embeddings

Third-party APIs

Canvas LMS
courses, assignments, grades

Google Classroom
alternate LMS provider

Google Gemini API
managed LLM alternative

Web Search API
industry-trend grounding

GitHub Actions
build, test, push, deploy

this one box is the whole of View 2

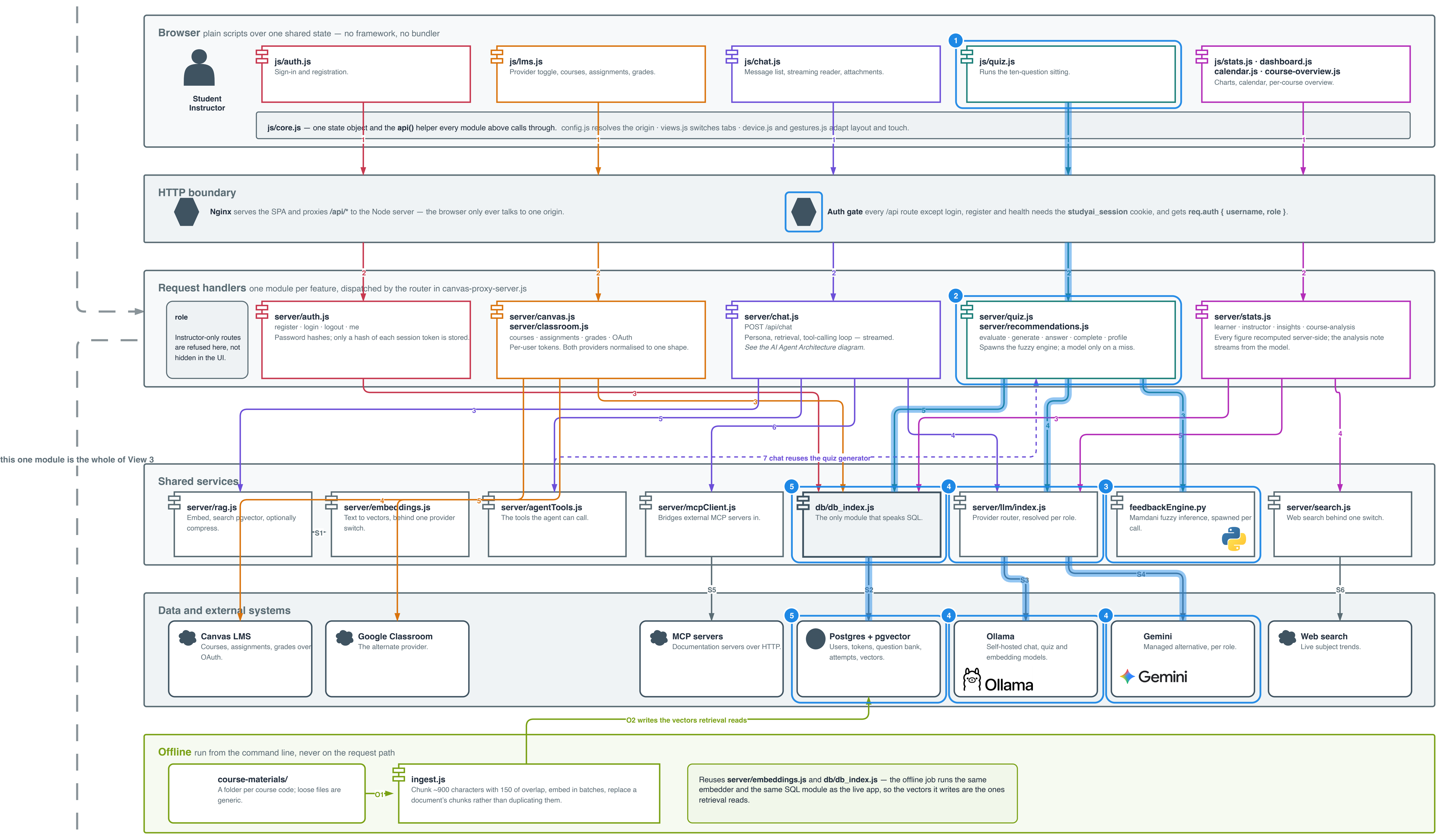
How to read this diagram
Two numbered sequences. **1–10** on the solid arrows trace one student request end to end; double heads mean the response returns the same way, and chat replies stream back as Server-Sent Events. **S1–S8** on the dashed arrows are supporting and scheduled traffic that no single request waits on — deploys, secrets, logs, cache and the nightly ingest.

Merged from the two drafts
DynamoDB, Aurora and MongoDB collapse into one Aurora PostgreSQL cluster — pgvector makes it the relational store and the vector store at once.
Lambda-behind-API-Gateway becomes ECS Fargate behind an ALB, because chat streams responses and holds a tool-calling loop open past a Lambda timeout. Lambda keeps the job it suits: scheduled ingest.

Production target vs. the prototype running today
Aurora PostgreSQL + pgvector ← today a pgvector/postgres:16 container on one EC2 host. Same schema, same queries.
ECS Fargate service ← today the same backend image as a Docker container on that host; Nginx serves the SPA instead of CloudFront + S3.
EC2 GPU Auto Scaling group ← today Ollama runs on a developer Mac, published to the EC2 host over a reverse SSH tunnel. Gemini is already wired as the managed alternative, selectable per role.
Cognito user pool ← today application-managed sessions (users / user_sessions tables) with cookie auth on every /api route.
EventBridge + Lambda ingest ← today ingest.js, run manually against local course-material files.
CloudFront, WAF, ALB, NAT, ElastiCache, Secrets Manager, multi-AZ ← production hardening, not provisioned in the prototype.

View 2 · Application components

Which module calls which. Each feature carries its own 1, 2, 3 … in its own colour; S1–S6 are shared, O1–O2 offline.



Browser plain scripts over one shared state — no framework, no bundler

Student
Instructor

js/auth.js
Sign-in and registration.

js/lms.js
Provider toggle, courses, assignments, grades.

js/chat.js
Message list, streaming reader, attachments.

1 **js/quiz.js**
Runs the ten-question sitting.

js/stats.js · dashboard.js
calendar.js · course-overview.js
Charts, calendar, per-course overview.

js/core.js — one state object and the **api()** helper every module above calls through. config.js resolves the origin · views.js switches tabs · device.js and gestures.js adapt layout and touch.

HTTP boundary

Nginx serves the SPA and proxies **/api/*** to the Node server — the browser only ever talks to one origin.

Auth gate every /api route except login, register and health needs the **studyai_session** cookie, and gets **req.auth { username, role }**.

Request handlers one module per feature, dispatched by the router in canvas-proxy-server.js

role

Instructor-only routes are refused here, not hidden in the UI.

server/auth.js
register · login · logout · me
Password hashes; only a hash of each session token is stored.

server/canvas.js
server/classroom.js
courses · assignments · grades · OAuth
Per-user tokens. Both providers normalised to one shape.

server/chat.js
POST /api/chat
Persona, retrieval, tool-calling loop — streamed.
See the AI Agent Architecture diagram.

2 **server/quiz.js**
server/recommendations.js
evaluate · generate · answer · complete · profile
Spawns the fuzzy engine; a model only on a miss.

server/stats.js
learner · instructor · insights · course-analysis
Every figure recomputed server-side; the analysis note streams from the model.

7 chat reuses the quiz generator

this one module is the whole of View 3

Shared services

server/rag.js
Embed, search pgvector, optionally compress.

S1

server/embeddings.js
Text to vectors, behind one provider switch.

server/agentTools.js
The tools the agent can call.

server/mcpClient.js
Bridges external MCP servers in.

5 **db/db_index.js**
The only module that speaks SQL.

4 **server/llm/index.js**
Provider router, resolved per role.

3 **feedbackEngine.py**
Mamdani fuzzy inference, spawned per call.

server/search.js
Web search behind one switch.

S5 S2 S3 S4 S6

Data and external systems

Canvas LMS
Courses, assignments, grades over OAuth.

Google Classroom
The alternate provider.

MCP servers
Documentation servers over HTTP.

5 **Postgres + pgvector**
Users, tokens, question bank, attempts, vectors.

4 **Ollama**
Self-hosted chat, quiz and embedding models.
Ollama

4 **Gemini**
Managed alternative, per role.
Gemini

Web search
Live subject trends.

O2 writes the vectors retrieval reads

Offline run from the command line, never on the request path

course-materials/
A folder per course code; loose files are generic.

O1

ingest.js
Chunk ~900 characters with 150 of overlap, embed in batches, replace a document's chunks rather than duplicating them.

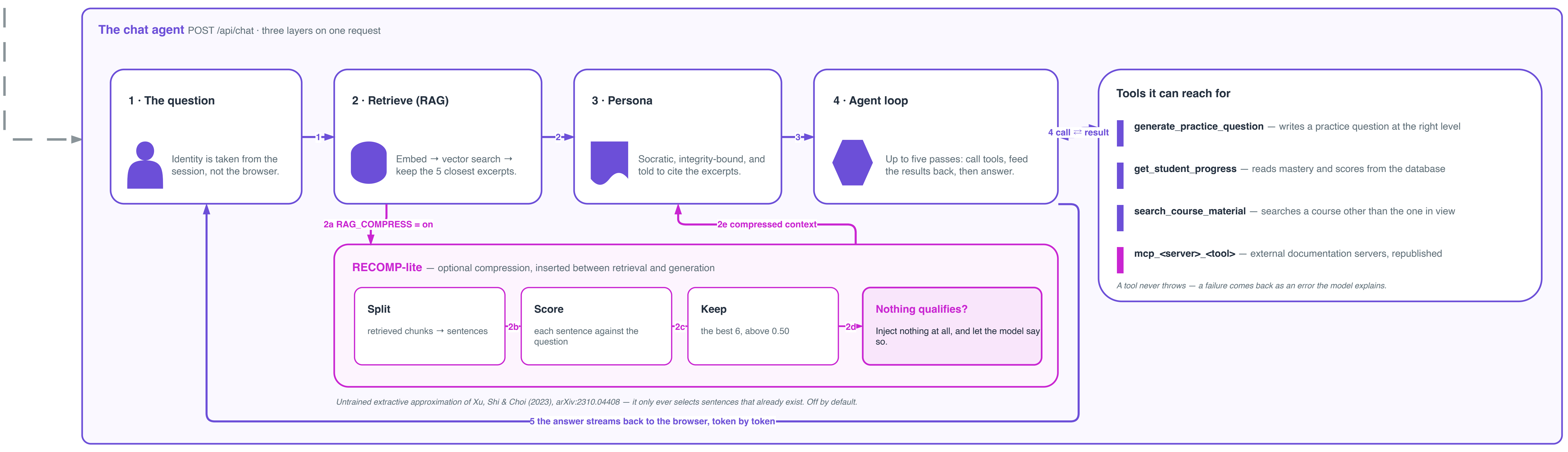
Reuses **server/embeddings.js** and **db/db_index.js** — the offline job runs the same embedder and the same SQL module as the live app, so the vectors it writes are the ones retrieval reads.

🟥 **Auth** 🟧 **Courses / LMS** 🟪 **Chat** 🟩 **Quiz** 🟪 **Insight**

Each feature carries its own 1, 2, 3 … — pick a colour and follow it from the browser down to the service it needs. **S1–S6** are the shared wires every feature depends on; **O1–O2** are the offline job. The one dashed wire is the only loop back up: the chat agent reuses the quiz generator rather than owning a second one.
Every shared service is loaded through server/services.js inside its own try/catch — one that will not load disables its feature and nothing else. The server still starts.

View 3 · AI agent

What happens inside server/chat.js on one turn. Arrows 1–5, with 2a–2e for the optional compression detour.



The chat agent POST /api/chat · three layers on one request

1 · The question
Identity is taken from the session, not the browser.

1

2 · Retrieve (RAG)
Embed → vector search → keep the 5 closest excerpts.

2

3 · Persona
Socratic, integrity-bound, and told to cite the excerpts.

3

4 · Agent loop
Up to five passes: call tools, feed the results back, then answer.

4 call ⇄ result

Tools it can reach for
generate_practice_question — writes a practice question at the right level
get_student_progress — reads mastery and scores from the database
search_course_material — searches a course other than the one in view
mcp_<server>_<tool> — external documentation servers, republished
A tool never throws — a failure comes back as an error the model explains.

2a RAG_COMPRESS = on

2e compressed context

RECOMP-lite — optional compression, inserted between retrieval and generation

Split
retrieved chunks → sentences

2b

Score
each sentence against the question

2c

Keep
the best 6, above 0.50

2d

Nothing qualifies?
Inject nothing at all, and let the model say so.

Untrained extractive approximation of Xu, Shi & Choi (2023), arXiv:2310.04408 — it only ever selects sentences that already exist. Off by default.

5 the answer streams back to the browser, token by token

What the agent runs on every lane on this page resolves through here

4 **Provider router**
Chosen per role — chat, quiz and embeddings can each sit on a different provider.

4 **Self-hosted**
Ollama
llama3.2 for the agent, bge-m3 for embeddings

4 **Managed**
Gemini
A drop-in swap for any generation role

5 **Postgres + pgvector**
Course-material vectors, the question bank, and every attempt

MCP servers
Documentation servers, reached over HTTP

Web search
Live subject trends for the instructor note

! **Nothing above is allowed to fail the request.** Retrieval that breaks returns no context and the turn drops to the persona alone. A tool that fails returns { error } and the model explains it. No search key simply disables search. No model reachable falls back to a rules-written note. The feature degrades; the request still answers.

Three more workflows on the same foundations

3 **Adaptive quiz**
Fuzzy inference picks the target; the model fills the gap.

1. **Running score** an exponential moving average of the result
2. **Fuzzy engine** maps it to a Bloom level and a complexity band
3. **Pick or generate** bank first, model only on a miss — then cached
4. **Coaching note** written once, for the whole sitting

6

Instructor · Analyze this course
Evidence from the database, grounded against live sources.

1. **Class signals** accuracy, hardest questions, recurring themes
2. **Live web search** current trends in the subject, in parallel
3. **One prompt** both, plus coursework titles from the client
4. **Streamed note** the same wire format the chat uses

Offline RAG ingestion
Runs ahead of time, never on the request path.

1. **Course materials** a folder per course code
2. **Chunk** ~900 characters, 150 of overlap
3. **Embed** the same model the chat path uses
4. **Store** vectors with an HNSW index, replaced not duplicated